

FOUNDATIONS AND APPLICATIONS OF BLOCKCHAINS

Homework 1

Depto. de Computación, 2^{do} Cuatrimestre 2023

24/8/23

Student:

Due: 25/9/23, 3:30pm

Instructions

- The way to deliver your solution will be explained in the following classes.
If you are proficient with $\text{\LaTeX}$, you may also typeset your submission and submit in PDF format. To do so, uncomment the “`%\begin{solution}`” and “`%\end{solution}`” lines and write your solution between those two command lines.
- Your solutions will be graded on *correctness* and *clarity*. You should only submit work that you believe to be correct.
- You may collaborate with others on Questions 1-3 of this problem set. However, you must **write up your own solutions** and **list your collaborators and any external sources** for each problem. Be ready to explain your solutions orally to a member of the course staff if asked. You **may not** collaborate with others on Question 4.

This homework contains 4 questions, for a total of 60 points.

1. We saw in class the notion of *digital signatures* and their security properties, *existential unforgeability* being an important one.

(a) (5 points) Describe the purpose of a *Public-Key Infrastructure* (PKI). Can the security properties of a digital signature be guaranteed without a PKI? Elaborate.

Solution: El propósito de PKI es proveer a los distintos nodos de una red una manera de verificar la autenticidad de los mensajes enviados por otros participantes. En particular, en clase vimos el problema de los generales bizantinos (la version de que uno le envia a todos un mensaje). Si ese mensaje estuviese firmado, podríamos verificar que viene realmente de ese "general bizantino". Pero para verificar la clave pública de ese general, necesitamos que esté autorizado por una CA (*Certificate Authority*).

Las propiedades de una firma digital son las siguientes:

- Autenticidad: el mensaje fue enviado por quien lo firmó.
- Integridad: el mensaje no fue modificado.
- No repudio: el firmante no puede negar que lo firmó.
- *Existential unforgeability*: un atacante no puede crear otra firma válida aunque conozca otras firmas de ese mismo firmante.

Para cumplir la autenticidad sin PKI, necesitaríamos que cada nodo comparta con los demás su clave pública de una manera confiable. Esto es para que, cuando reciba una firma, pueda verificar que coincide con la verdadera clave pública del firmante. Esto asumiría que la clave pública que tengo de ese participante es la correcta. Con PKI sería mas simple, ya que cada nodo puede verificar la clave pública de otro nodo a través de la CA.

El no-repudio es particularmente problemático, ya que el firmante podría simplemente alegar que esa no es su clave pública. Si tuviésemos PKI, esta entidad podría intervenir y decir que esa no es la clave pública del firmante. La integridad y *existential unforgeability* se pueden cumplir sin PKI, ya que dependen del algoritmo de firma en sí mismo.

(b) (5 points) Bitcoin transactions use the ECDSA signature scheme. Does Bitcoin assume a PKI? If not, reconcile with the above argument.

Solution: Bitcoin no asume un PKI. La razón es porque cada participante de una transacción se identifica por su "dirección", que es justamente su clave pública. Por ende, el hecho de el mensaje esté firmado por esa dirección nos permite verificar que fue enviado por el dueño.

Si la firma no es válida, la transacción se va a rechazar por los nodos de la red. Esto garantiza que solamente quien tiene la clave privada puede realizar transacciones. En relación a lo dicho anteriormente, vemos que el no repudio se cumple en el sentido de que quien envió una transacción no puede negar haber tenido la clave privada de esa dirección. En cuanto a la autenticación de que una transacción fue enviada por el dueño de la dirección, vemos que esto se cumple por lo explicado anteriormente: la clave pública es justamente la dirección de quien envía los fondos.

2. (10 points) Refer to Algorithm 4 (Main Loop) in [GKL15]. We saw in class that blockchains would start from a “genesis” block, which would provide an unpredictable string to start mining from. Yet, notice that in Algorithm 4 mining starts from the empty string ($\mathcal{C} \leftarrow \varepsilon$). How come? Explain why this works.

Solution: Vemos que al inicio del algoritmo ($\mathcal{C} \leftarrow \varepsilon$). En la ronda número 2, vale que $init = False$, entonces se entra a la rama else del if.

Luego, se busca la máxima cadena válida entre las disponibles, incluyendo $\mathcal{C} = \varepsilon$. Asumimos que no hay otras disponibles en $RECEIVE()$, entonces que $\tilde{\mathcal{C}} = \varepsilon$, pues es válida.

Vemos que es válida observando el código de *validate*, en el que se retorna $b = V(\mathbf{x}_{\mathcal{C}})$. Por convención (dicho en el paper), vemos que $\mathbf{x}_{\mathcal{C}}$ de una cadena vacía es siempre 1. Por ende, la cadena vacía es válida.

Luego, se genera el valor x con la función $I(\cdot)$ y $\mathcal{C}_{new}$ con *pow*. Vemos que esta $\mathcal{C}_{new}$ generada va a ser el resultado de que se tome $s = 0$ en *pow*, y que se agregue un nuevo bloque a $\tilde{\mathcal{C}}$. Este nuevo bloque va a ser $\langle s, x, ctr \rangle$, con $s = 0$. El x y ctr van a ser los que cumplan con $H(ctr, h) < T$, siendo $h = G(0, x)$

Luego, obtuvimos una nueva cadena ($\mathcal{C} \leftarrow \mathcal{C}_{new}$) con dos bloques, el vacío y el que generamos recién. Las rondas subsiguientes entrarían en los casos en los que se van a usar otras cadenas (no la vacía) como $\tilde{\mathcal{C}}$, ya que $\mathcal{C}$ contendrá un bloque válido.

3. Refer to Algorithm 1 (validate) in [GKL15], which implements the *chain validation predicate*.

(a) (5 points) Rewrite validate so that it starts checking from the *beginning* of the chain.

Solution:

Tomé en cuenta que, en cada bloque, s representa el hash del bloque anterior. Por eso uso previousHash, que nota el hash del bloque anterior para ser verificado con el siguiente. Otra cosa a tener en cuenta es que usé first para obtener el primer bloque de la cadena, y $\mathcal{C}^{1\downarrow}$ para obtener toda la cadena excepto el primer bloque.

```
1: function VALIDATE( $\mathcal{C}$ )
2:    $b \leftarrow V(x_{\mathcal{C}})$ 
3:   if  $b \wedge \mathcal{C} \neq \varepsilon$  then
4:      $\langle s, x, ctr \rangle \leftarrow \text{first}(\mathcal{C})$ 
5:     if NOT ( $\text{validblock}_q^T(\langle s, x, ctr \rangle)$ ) then
6:        $b \leftarrow \text{False}$ 
7:     else
8:       previousHash  $\leftarrow H(ctr, G(s, x))$ 
9:        $\mathcal{C} \leftarrow \mathcal{C}^{1\downarrow}$ 
10:      while  $\mathcal{C} \neq \varepsilon \wedge b = \text{True}$  do
11:         $\langle s, x, ctr \rangle \leftarrow \text{first}(\mathcal{C})$ 
12:        if  $\text{validblock}_q^T(\langle s, x, ctr \rangle) \wedge (s = \text{previousHash})$  then
13:          previousHash  $\leftarrow H(ctr, G(s, x))$ 
14:           $\mathcal{C} \leftarrow \mathcal{C}^{1\downarrow}$ 
15:        else
16:           $b \leftarrow \text{False}$ 
17:        end if
18:      end while
19:    end if
20:  end if
21:  return  $b$ 
22: end function
```

(b) (5 points) Discuss pros and cons of this approach, compared to Algorithm 1's.

Solution: La principal desventaja de este algoritmo es que, si queremos chequear la validez de una cadena muy larga, y que probablemente si hay un error, este en los últimos bloques, vamos a tardar mucho hasta llegar hasta ahí, ya que estamos mirando desde adelante hacia atrás. Decimos que es más probable que haya un error en los últimos bloques ya que, en el contexto de

una blockchain real, los bloques se agregan al final, y los nodos comparten una gran parte de la cadena inicial (que si los nodos son honestos debería ser válida). Por lo tanto, tiene sentido comenzar a validar desde el final.

Otra desventaja es que es un poco más difícil de entender. Como las cadenas están formadas por bloques que apuntan a su bloque anterior, sería intuitivo ir desde adelante hacia atrás. En este caso estamos yendo desde atrás hacia adelante.

Una ventaja que tiene el algoritmo que va hacia adelante es que si, por alguna razón hubiera errores en los primeros bloques, los detectaríamos antes. También puede ser mas fácil de entender el hecho de avanzar hacia adelante, pero una vez que se comprende la estructura de las cadenas y los bloques, queda claro que ir hacia atrás es hacer mejor uso de las estructuras de datos.

4. **Smart contract programming:** *Matching Pennies*. This assignment will focus on writing your own smart contract to implement the Matching Pennies game. The contract should allow two players (A, B) to play a game of Matching Pennies at any point in time. Each player picks a value of two options—for example, the options might be {0, 1}, {'a', 'b'}, {True, False}, etc. If both players pick the same value, the first player wins; if players pick different values, the second player wins. The winner gets 0.1 ETH as reward. After a game ends, two different players should be able to use your contract to play a new game.

Example: Let A, B be two players who play the game, each with 1 ETH. A picks 0 and B picks 0, so A wins. After the game ends, A's balance is 1.1 ETH (perhaps minus some gas fees, if necessary).

You should implement the smart contract and deploy it on the course's Goerli Testnet. Your contract should be as *secure*, *gas efficient*, and *fair* as possible. After deploying your contract, you should engage with other student's contract and play a game on his/her contract. Before you engage with a fellow student's smart contract, you should evaluate their code and analyze its features in terms of security and fairness (refer to Lectures 07 and 08). You should provide:

- (a) (10 points) The code of your contract, together with a detailed description of the high-level decisions you made for the design of your contract, including:
- Who pays for the reward of the winner?
 - How is the reward sent to the winner?
 - How is it guaranteed that a player cannot cheat?
 - What data type/structure did you use for the pick options and why?

Solution: Dejo acá el código fuente del contrato. Si lo quieren leer un poco mejor, dejo el link del archivo en github que uso para que lo puedan leer con linter.

```
// SPDX-License-Identifier: GPL-3.0

// This makes sure we use a solidity version that
// automatically handles overflow/underflow for me :D.
// It reverts the operation if it overflows/underflows.
pragma solidity ^0.8.0;

contract MatchingPennies {
    address public playerA;
    address public playerB;
    bool public valueA;
    bool public revealedA;
    bool public valueB;
```

```

bool public revealedB;
bytes32 public commitmentA;
bytes32 public commitmentB;
uint256 public constant REWARD = 0.1 ether;
mapping(address => uint256) public balances;

// Example:
// bytes32 commitment = keccak256(
//   abi.encode(true, "mySecret123")
// );
// commitPlay(commitment) {value: 0.05 ether}
function commitPlay(bytes32 commitment) public payable {
    require(
        msg.value == REWARD / 2,
        "Must pay half the reward to play"
    );

    // if A hasn't played yet
    if (playerA == address(0)) {
        playerA = msg.sender;
        commitmentA = commitment;
        return;
    } else if (playerB == address(0)) {
        require(
            msg.sender != playerA,
            "Player A cannot play again"
        );
        playerB = msg.sender;
        commitmentB = commitment;
        return;
    } else {
        revert("Game already started");
    }
}

// Example:
//   revealPlay(my_choice, "mySecret123"),
//   where "mySecret123" is the secret used in commitPlay
function revealPlay(bool value, string memory secret) public {
    require(
        playerA != address(0) && playerB != address(0),

```

```

        "Both players must have played"
    );

    bytes32 hash = keccak256(abi.encode(value, secret));

    if (msg.sender == playerA) {
        require(!revealedA, "Player A has already revealed");
        require(
            hash == commitmentA,
            "Commitment does not match"
        );

        valueA = value;
        revealedA = true;
    } else if (msg.sender == playerB) {
        require(!revealedB, "Player B has already revealed");
        require(
            hash == commitmentB,
            "Commitment does not match"
        );

        valueB = value;
        revealedB = true;
    } else {
        revert("Not a player");
    }

    if (revealedA && revealedB) {
        determineWinner();
    }
}

function determineWinner() private {
    require(
        revealedA && revealedB,
        "Both players must have revealed"
    );

    address winner;
    if (valueA == valueB){
        winner = playerA;
    }
}

```

```

    } else {
        winner = playerB;
    }

    // clean up the state
    playerA = address(0);
    playerB = address(0);
    revealedA = false;
    revealedB = false;
    valueA = false;
    valueB = false;
    commitmentA = bytes32(0);
    commitmentB = bytes32(0);

    // have the winner be able to retrieve the funds later.
    balances[winner] += REWARD;
}

function getBalance() public view returns (uint256) {
    return balances[msg.sender];
}

function withdraw() public {
    uint256 balance = balances[msg.sender];
    require(balance > 0, "No balance to withdraw");
    balances[msg.sender] = 0;
    payable(msg.sender).transfer(balance);
}

// Idea: agregar una funcion 'forfeit' (rendirse)
// que puede ser llamada por A para retirar sus fondos si
// no hubo ningun jugador B que se haya unido a la partida.
// Esta funcion chequearia que
// el sender sea A, que B no haya jugado
// y le dejaria a A sus fondos en 'balances[playerA]'
}

```

Explicación del contrato:

El premio del ganador lo paga quien perdió. Es decir, ambos jugadores (A y B) pagan la mitad del premio al participar (en el método `commitPlay`) Para este método los participantes deben enviar un commitment de su elección.

Este commitment lo generan con

```
bytes32 commitment = keccak256(abi.encode(true, "mySecret123"));
```

Junto con el commitment, deben enviar exactamente 0.05 ether.

Luego, una vez que el segundo jugador ya jugó, el primer jugador puede revelar su elección con `revealPlay(my_choice, "mySecret123")`.

Ahí se tienen que enviar los mismos valores de `my_choice` y `"mySecret123"` que se usaron antes. Una vez que *B* revela su elección, se determina el ganador, que puede retirar sus fondos usando el método `withdraw`.

Se garantiza que un jugador no pueda hacer trampas por los siguientes motivos:

- Ningún jugador puede ver la elección del otro jugador, ya que lo que se envía es un commitment. Si no fuese así, el segundo jugador siempre estaría en ventaja, pues podría ver en la blockchain la elección de *A* y elegir lo opuesto para ganar.
- Las elecciones no pueden ser reveladas si ambos jugadores no `commit`earon aún.
- Se garantiza que un jugador no pueda cambiar su elección una vez que envió el `commit`, pues no podría haber obtenido el mismo hash con otra elección.

Para la elección usé un booleano, ya que me pareció lo más simple dado que solo hay dos opciones. Guardé otro booleano para cada una de las elecciones, que me dice si el valor que tengo guardado en `valueA` o `valueB` fue efectivamente revelado.

Otra opción podría haber sido guardar un entero, con valores 0, 1 y 2. El 0 representaría que no se ha revelado la opción aún. El 1 sería `true` y el 2 sería `false`.

- (b) (5 points) A detailed gas consumption evaluation of your implementation, including:
- The cost of deploying and interacting with your contract.
 - Whether your contract is *fair* to both players, including whether one player has to pay more gas than the other and why.
 - Techniques to make your contract more cost efficient and/or fair.
- (c) (5 points) A thorough list of potential hazards and vulnerabilities that *may* occur in your contract; provide a detailed analysis of the security mechanisms you use to mitigate such hazards.
- (d) (5 points) A description of your analysis of your fellow student's contract (along

with relative code snippets of their contract, where needed for readability), including:

- Any vulnerabilities discovered?
- How could a player exploit these vulnerabilities to win the game?

(e) (5 points) The transaction history of an execution of a game on your contract.

